RMIT UNIVERSITY

Report

Introduction to ROS2

Dinh-Son Vu

School of Science, Engineering, and Technology

Table of Contents

1. Introduction

This is an introduction to ROS2, which is a middleware running on a development computer or an on-board computer (Raspberry Pi, Jetson). ROS2 comes with a steep learning curve, but it is useful to build robots with intelligence, contrary to robots that are teleoperated only.

- **ROS2** stands for Robotic Operating System, and it is the second version. The term Operating system is quite confusing because ROS2 does not include a graphical interface just like Windows or MacOS, as it mainly uses terminal command. The first version of ROS is still quite widely used, but does not scale well with robot fleet, hence the development of ROS2.
- **ROS2 is a middleware**: it is the part that stands in between the hardware and the software. ROS2 is similar to a driver for peripherals: e.g., when you plug a mouse to a computer, it just works instantaneously. The drivers are installed automatically as soon as the device is plugged in, and you do not write any code to convert the optical flow sensor (mouse sensor) to movement in the x-y direction. Your computer understands the information provided by the mouse directly thanks to the standardization of the communication protocol via USB.

1.1. Challenges regarding autonomous robots

Please watch at the videos developed by Norlab, in French, but with English subtitles available. The laboratory Norlab focuses on autonomous robots in northern, harsh environment.

- History of autonomous vehicles: https://youtu.be/-vKPvJXooNs?si=iUMQm3fQmWnBT1B5
- What is a mobile robot? https://youtu.be/PJn7bch13Yo?si=kv2qXKYrSWB_zeJb

A robot needs sensors to perceive the environment, algorithms to treat these information, and actuators to act accordingly.

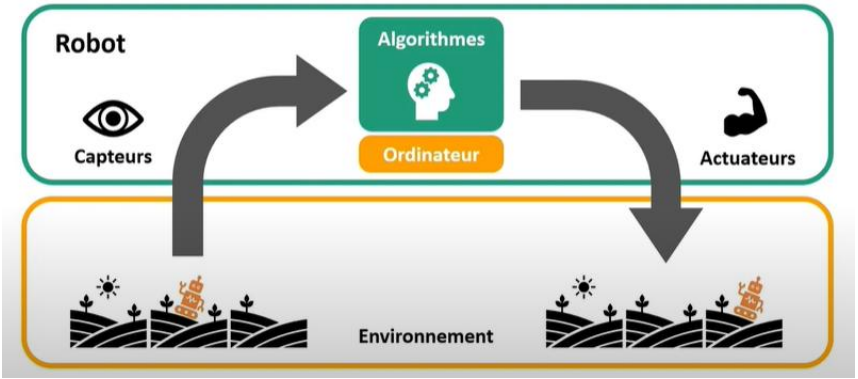

Figure 1: An autonomous robot requires sensors, actuoators, and algorithms

A robot has several peripherals (generic names for sensors and actuators, similar to peripherals for a computer, e.g., mouse, keyboard, screen, speakers, microphone, etc.), with different algorithms that are intense and quite consuming to develop, such as:

- Code interfacing between the hardware to the computer (i.e., a driver)

  - Sensors: Lidar, radar, sonar, camera, GPS, IMU, encoder
  - Actuators: DC, BLDC, stepper
- Algorithms to interpret the sensors' data:
  - Computer vision with camera sensor
  - Mapping/SLAM with Lidar, radar, sonar sensor
  - Positioning/navigation with GPS, IMU, and encoder
  - Digital twin – Visualization/simulation of the robot on the computer

For simple tasks, such as teleoperation of a wheeled robot, Arduino/esp32 are fine. Command from the joystick are translated to command to the robots' motors. However, complex tasks involve more hardware, communication protocol, and computation. For instance, a typical autonomous robot involve four wheels, four encoders, one depth camera with computer vision algorithm, lidar with mapping (slam), swarm/decentralized communication for multiple robots, texture mapping with depth 3D camera …: there is no way to implement this into an Arduino robot.

1.2. ROS2, middleware between software and hardware

In French, with English subtitles available.

- Why ROS2? https://youtu.be/vAb5SnaJbF0?si=bhGIA70abbXpxJWM
- Camera and vision: https://youtu.be/hfCB2lRdQmg?si=FS-GwnXfUbptW-K2
- Modeling and control: https://youtu.be/UhVlXqckmv4?si=St-mZjxOBteLqQ2Q
- Lidar and SLAM: https://youtu.be/nqcmNkzTbM4?si=djCLtcyxlNUTXqgH

The development of robotic systems is a worldwide activity. Thus, robotic engineers share common problems, such as drivers for hardware, navigation, SLAM, computer vision, etc.

- ROS2 is based on the systems of **nodes** (pieces of individual code) that are communicating together with channels called **topics.**
- It uses two programming languages: Python for prototyping, analysis, and communication between nodes. The other language C++ is used for real-time (microcontroller).
- The greatest benefit of ROS2 is code sharing: you can decide to write down the computer vision algorithm all over again, or to use a library that has already been developed: it requires works regarding the inputs, outputs, and parameters of the packages, but it is much less prone to errors relative to coding the algorithms by yourself.

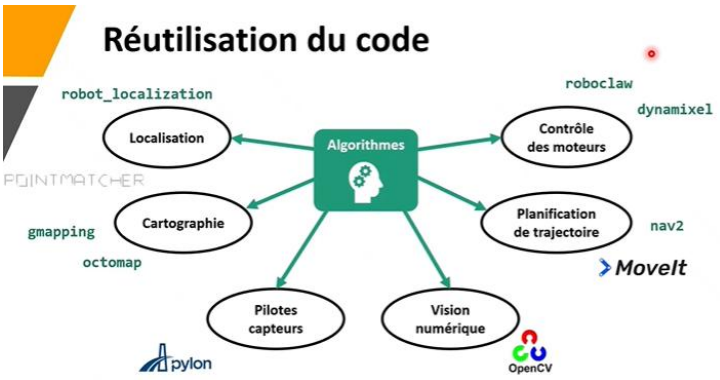

Figure 2: Autonomous vehicles must handle several tasks: mapping, navigation, computer vision, etc. Code sharing become even more important as the robot is becoming more autonomous and intelligent.

Figure 3: ROS2 aims at creating a standard for all robotic platforms: they thus can be summarized as black box receiving inputs and transmitting outputs.

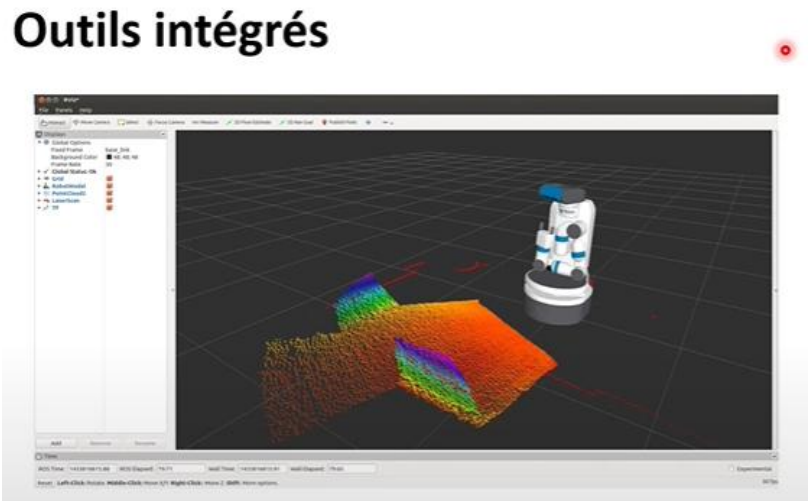

Figure 4: Digital Twin (Rviz and Gazebo) - ROS2 offers tools for visualization of data coming from the hardware, which is time consuming if one needs to develop it entirely.

1.3. ROS2 main concept

ROS2 aims at creating a standard for different hardware, software, and engineers. Thus the ROS2 framework should be understood first.

- **Node**: it is a bloc of code. For example, inverse kinematics takes Cartesian position x,y,z as an input and output joint angles.
- **Topic**: Data, e.g., encoders, imu, lidar, etc. Nodes subscribe to a topic (it receive information from the topic as an input) or publish to a topic (it outputs the topic).
- ROS2 uses the **Terminal** extensively: one must become comfortable with command line.
- **URDF** stands for Unified Robot Description Format: it describes the robot dimension and types of joint in an xml format. This is an essential part of ROS2, as one must perform simulation and visualization of the actual robot.
- **Packages** is a collection of nodes, topics, and libraries that performs a specific task, e.g., computer vision package, SLAM package, etc. Thus, one must understand the concept of cloning, compiling, and utilizing a package.
- **Launch files** is a Python script to launch of the required packages for your robot.

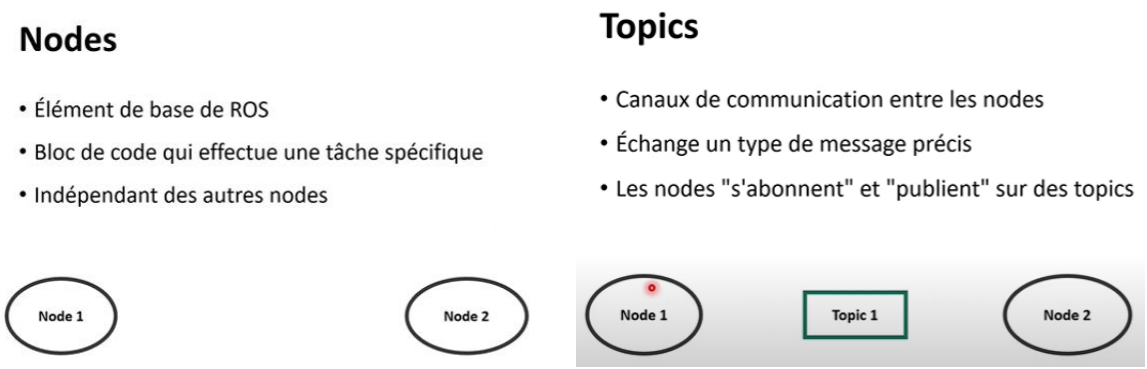

Figure 5: Nodes and Topics are the cornerstone of ROS2.

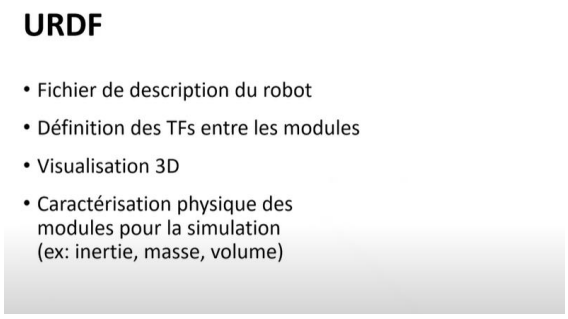

Figure 6: URDF is essential for describing a robot in ROS2 and especially for RVIZ

Figure 7: ROS2 is about integrating packages together

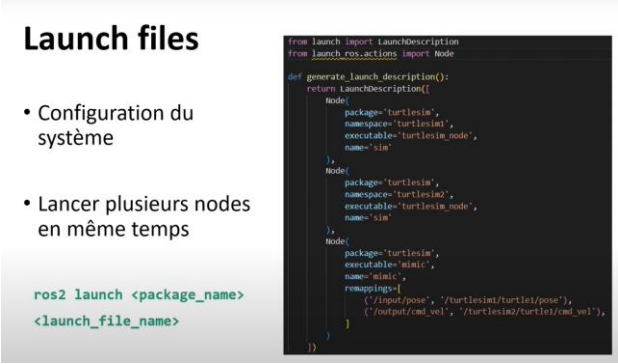

Figure 8: Launch file: a bit similar to MATLAB script: you launch the nodes

1.4. ROS2: Packages for autonomous robots

The following GitHug has an excellent architecture regarding the code architecture. The generic name used for the robot in this document is rmitbot. Feel free to modify its name. Some of the tools are not presented yet, but will come in the upcoming sections.

https://github.com/AntoBrandi/Bumper-Bot

https://github.com/ros-mobile-robots/diffbot (this is ROS1 though)

https://automaticaddison.com/naming-and-organizing-packages-in-large-ros-2-projects/

https://github.com/aws-deepracer/aws-deepracer/blob/main/introduction-to-the-ros-navigation-stack-using-aws-deepracer-evo.md

Name	Description
1. rmitbot_description	- contains urdf and gazebo configuration - uses gazebo and rviz (package for visualization) - The virtual robot cannot move yet.
2. rmitbot_controller	- uses ros2_control and ros2_controller (package for controlling robot) - interface the joystick/keyboard controller to the joints of the robot - the virtual robot can move, its motion can be measured with odometry. Adding an IMU would attenuate the error due to slippage.
3. rmitbot_localization	- uses robot_localization (package for ekf) - performs sensor fusion (ekf) between odometry and imu - the virtual robot can move, and its pose can be displayed
4. rmitbot_mapping	- Create a map of the environment with slam_toolbox
5. rmitbot_navigation	- path planning (a_star): path to get to the target - motion planning: path following with PID - navigation: decision tree relative to known and unknown obstacles
6. rmitbot_vision	- includes camera plugins
7. rmitbot_controller_hardware	- controlling the motors
8. rmitbot_localization_hardware	- getting the imu information
9. rmitbot_mapping_hardware	- getting lidar information
10. rmitbot_navigation_hardware	- moving the robot autonomously
11. rmitbot_vision_hardware	- getting camera feed

The workspace tree should look like the following:

rmitbot_ws

- build, install, log, .vsode: → Folders are automatically created while setting up the ws
- src → contains all the packages listed below
    - rmitbot_bringup
    - rmitbot_description
    - rmitbot_controller
    - rmitbot_localization
    - rmitbot_mapping
    - rmitbot_navigation
    - rmitbot_firmware
    - rmitbot_vision

2. Getting started with ROS2

There are several concepts to understand:

- Create a workspace and a package
- Push code on Github
- Clone code from Github

Even though creating a package is not mandatory, it will be beneficial to know the concept for developing your own code in the future.

2.1. Creating a workspace and a package

Install the compiler colcon:

```
sudo apt install python3-colcon-common-extensions
```

A workspace is just a folder that will contains packages. Make a directory called lesson1_ws, with a folder called src:

```
mkdir -p lesson1_ws/src
```

Packages are stored in the src folder. The nodes of a package can be code in python or cpp. For the moment, we will focus on python code. Change directory to the src folder and build a new package called my_package:

```
cd lesson1_ws/src
ros2 pkg create --build-type ament_python my_package → for python nodes
ros2 pkg create --build-type ament_cmake my_package → for cpp nodes (for reference)
```

A folder called “my_package” has been created, with the following components: include, src, CMakeLists.txt, package.xml. Verify that your package is available

```
ros2 pkg list
```

For the moment, the newly package created does nothing (there is no node/code insde).

Change directory back to the workspace folder and build the workspace It will create three folders: build, install, and log. Then source the workspace, to let the computer know where you are starting to work on your computer.

```
cd ~/lesson1_ws
colcon build
source install/setup.bash        → source the workspace
```

2.2. Creating a launch file

Tutorial: https://youtu.be/xJ3WAs8GndA?si=W_VEgCKo5m_yVBqb

Allows to launch several nodes with their associated parameters.

Go to the src of your workspace and create a package. You may change the name of the robot:

```
ros2 pkg create my_awesome_robot
```

You may remove include and src folder, since you are not writing a node

```
rm -rf include/
rm -rf src/
```

Create a launch folder:

```
mkdir launch
```

Create a launch file in the launch folder

```
cd launch/
touch demo.lauch.py
```

Come back to the src folder and start VSCode

```
cd ../..
code .
```

Follow the step in the video to modify the launch file. Build the package from the workspace folder, source the workspace, and launch the launch file:

```
colcon build
source install/setup.bash
ros2 launch my_robot demo.launch.py
```

ros2 launch <package_name> <launch_file> ros2 launch gazebo_tutorial gazebo.launch.py	Launch a launch file (python)
ros2 pkg list ros2 pkg executables ros2 pkg executables <pkg_name>	Packages available in ros2 Executables (e.g., nodes) available in ros2 Executables of a particular package

Warning: you may need to install xterm to use teleop keyboard in a new terminal.

2.3. Make your first commit on Github

Make sure you have installed Git (please refer to the setup of your dev computer). Create a new repository on your GitHub account – for example lesson1_ws. Do not add a readme file yet.

On VSCode, go to your folder containing the code and additional folders:

- Initialization: git init
- Add everything: git add .
- Make a commit: git commit -m "first commit"
- Create a branch: git branch -M main
- Remote path (change the address to the repository you have created):
  - git remote add origin https://github.com/ROS2-AutoBot/lesson1_ws.git
- Sometimes, there is already an origin, check with
  - git remote -v
- You may change it with the following cli
  - git remote set-url origin https://github.com/ROS2-AutoBot/lesson1_ws.git
- Push it to GitHub: git push -u origin main

Normally, the entire folder should be copied on your online repository.

2.4. Clone code from GitHub to your computer

This is an essential task: since most of the code have been developed already, the aim is to clone the correct code and adapt it to your needs. In a terminal, navigate to the folder/workspace where you would like to clone the repository.

```
cd <workspace>
```

Clone the repository:

```
git clone https://github.com/ROS2-AutoBot/lesson1_ws.git
```

Change back to the workspace directory, compile the project, and source the workspace:

```
cd <workspace>
colcon build
. install/setup.bash
```

If there is a compiling error, you may have to remove build, install, log, and colcon build.

3. Plotjuggler for data visualization

Information: https://index.ros.org/p/plotjuggler/

Tutorial: https://youtu.be/MnMGjvYxlUk?si=3zabX6mJ8EmFIR0X

Install ros package: sudo apt install ros-jazzy-plotjuggler-ros

Launch plotjuggler: ros2 run plotjuggler plotjuggler

Example:

- In a first terminal, launch the teleoperation keyboard node:

ros2 run teleop_twist_keyboard teleop_twist_keyboard

- In a second terminal, launch plotjuggler:

ros2 run plotjuggler plotjuggler

- In the third terminal, type “ros2 node list”. You should see:
  /plotjuggler
  /teleop_twist_keyboard
- In the third terminal, type “ros2 topic list”. You should see:
  /cmd_vel → This is the topic from the node /teleop_twist_keyboard
  /parameter_events → This is by default used by ROS2
  /rosout → This is by default used by ROS2
- In plotjuggler, chose the correct topic to stream and to visualize the data.

Plotjuggler will be essential to record and measure the robot movement and include it in your final report.

4. Appendix

Useful information:

- Articulated Robotics: https://youtu.be/XcJzOYe3E6M?si=HZpo33TDFA1Cuja0
  https://articulatedrobotics.xyz/tutorials/docker/docker-overview
- F1 Tenth at UPenn: https://youtu.be/EU-QaO6xTv4?si=LN_v0i6MaEkdJbco